33. Queue in Java — Quick Notes

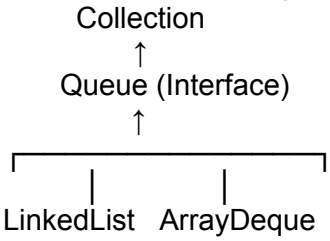
Queue follows **FIFO (First In First Out)** principle.
The element inserted first is removed first.

What is Queue?

First In → First Out (FIFO)
Real-life examples:

- Doctor's queue
- Ticket booking queue
- Printer queue
- Customer service queue

Queue Hierarchy



Queue Implementations

Implementation	Internal Structure	Preferred
`LinkedList`	Doubly Linked List	When worst-case O(1) is required
`ArrayDeque`	Dynamic Circular Array	✅ Generally Preferred

Why Prefer ArrayDeque?

Advantages

- Cache friendly
- Faster in practice
- Less memory overhead

Drawback

Insertion/Deletion: Amortized O(1)
One resize operation may take: O(n)

Queue Syntax

Using ArrayDeque (Recommended)

```
Queue<Integer> q = new ArrayDeque<>();
```

Using LinkedList

```
Queue<Integer> q = new LinkedList<>();
```

Basic Queue Operations

Operation	Method	Description
Insert	`offer(x)`	Add element at rear
Remove	`poll()`	Remove front element
Access Front	`peek()`	View front element
Size	`size()`	Number of elements
Empty Check	`isEmpty()`	Checks if queue is empty

Example

Queue<Integer> q = new ArrayDeque<>();

q.offer(10);
q.offer(20);
q.offer(30);

System.out.println(q);
Output: [10, 20, 30]

Queue Representation

After:
q.offer(10);
q.offer(20);
q.offer(30);

Queue becomes:

Front Rear
↓ ↓
10 → 20 → 30

Peek Operation

Returns the front element.
Does NOT remove it.
System.out.println(q.peek());

Output:10
Queue: 10 20 30

Poll Operation

Removes and returns front element.
System.out.println(q.poll());
Output: 10

Queue becomes: 20 30

Peek Again
System.out.println(q.peek());
Output: 20

Queue Methods Comparison

Operation	Safe Method	Exception Method	Failure Behavior
Insert	`offer(x)`	`add(x)`	`offer()` → false, `add()` → Exception
Remove Front	`poll()`	`remove()`	`poll()` → null, `remove()` → Exception
Access Front	`peek()`	`element()`	`peek()` → null, `element()` → Exception

Example
Queue<Integer> q = new ArrayDeque<>();

q.offer(10);
q.offer(20);

System.out.println(q.peek()); // 10
System.out.println(q.poll()); // 10
System.out.println(q.peek()); // 20

Output:
10
10
20

Queue Time Complexity

Operation	LinkedList	ArrayDeque
offer()	O(1)	O(1) Amortized
poll()	O(1)	O(1) Amortized
peek()	O(1)	O(1)
add()	O(1)	O(1) Amortized
remove()	O(1)	O(1) Amortized
element()	O(1)	O(1)
size()	O(1)	O(1)
isEmpty()	O(1)	O(1)

Amortized O(1)
For `ArrayDeque`: Most operations → O(1)

Occasionally: Internal resizing → O(n)
Average over many operations: Amortized O(1)

Queue Applications

1. CPU Scheduling

First Come First Serve (FCFS)

---

2. Web Servers

Incoming requests
↓
Queue
↓
Processing

---

3. Printer Queue

Print jobs processed
in arrival order.

---

4. Email Systems

Outgoing emails
stored in queue.

---

5. Network Routers

Packets arrive
↓
Queue
↓
Forwarded in order

---

6. Producer–Consumer Problems

Producer
↓
Queue
↓
Consumer

---

7. Buffers

Used when: Fast Device ↔ Slow Device
Example: Keyboard → Buffer → Processor

Interview Notes

// Queue follows FIFO.
// Insert at rear and remove from front.
// ArrayDeque is generally preferred over LinkedList
// because it is cache friendly and faster in practice.

Quick Revision Table

Topic	Remember
Principle	FIFO
Front Removal	poll()
Rear Insertion	offer()
View Front	peek()
Preferred Implementation	ArrayDeque
Queue Type	Interface
LinkedList Complexity	Worst-case O(1)
ArrayDeque Complexity	Amortized O(1)
Common Applications	Scheduling, Buffers, BFS

Most Common Queue Syntax

```
Queue<Integer> q = new ArrayDeque<>();

q.offer(10);
q.offer(20);
q.offer(30);

System.out.println(q.peek());   // 10
System.out.println(q.poll());   // 10
System.out.println(q.peek());   // 20
```

Output

```
10
10
20
```